

UNIVERSIDADE CATÓLICA DE BRASÍLIA

SISTEMA DE GESTÃO PARA CLÍNICA ODONTOLÓGICA

Artigo Técnico — Disciplina de Banco de Dados

Antônio Joaquim Alves · Cayo do Prado Santos
Dankley Meireles de Lima · Felipe de Sousa Lopes

Brasília – DF · 2026

RESUMO

Este artigo descreve o desenvolvimento de um sistema de gestão para clínica odontológica utilizando banco de dados relacional MySQL, com integração entre backend em Python (Flask) e frontend em HTML, CSS e JavaScript. O sistema contempla o gerenciamento de consultas, pacientes, dentistas, estoque de insumos e movimentações financeiras. A modelagem do banco de dados seguiu as etapas conceitual, lógica e física, resultando em dez tabelas relacionadas com uso de triggers, views, stored procedures, functions, índices e controle de acesso por usuários. A aplicação demonstra o domínio dos principais recursos de um SGBD em um contexto real e relevante para o Distrito Federal, onde há elevada concentração de profissionais odontológicos.

Palavras-chave: banco de dados relacional; MySQL; clínica odontológica; Flask; gestão de sistemas.

1. INTRODUÇÃO

O Distrito Federal apresenta um dos maiores índices de dentistas por habitante do Brasil. Segundo dados do Conselho Federal de Odontologia (CFO), há aproximadamente 9.343 cirurgiões-dentistas registrados no CRO-DF, resultando em uma proporção de 1 profissional para cada 326 habitantes — quatro vezes acima da recomendação da Organização Mundial da Saúde (OMS), que sugere 1 para cada 1.200 habitantes.

Diante desse cenário, muitos dentistas, especialmente os recém-formados, registram consultas e pacientes em papéis avulsos ou utilizam softwares dispendiosos e incompatíveis entre si. Com o objetivo de democratizar o acesso à tecnologia de gestão clínica, este trabalho propõe o desenvolvimento de um sistema integrado de gerenciamento para clínicas odontológicas, fundamentado em banco de dados relacional MySQL e acessível via interface web.

O sistema permite o cadastro e controle de pacientes, agendamento de consultas, gestão de estoque de insumos, controle financeiro e emissão de relatórios, promovendo eficiência operacional e segurança no armazenamento de dados clínicos.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um sistema de gestão para clínica odontológica com banco de dados relacional MySQL, integrando frontend e backend, demonstrando o domínio dos principais recursos de um SGBD.

2.2 Objetivos Específicos

- Modelar o banco de dados nas etapas conceitual, lógica e física, aplicando normalização e integridade referencial;
- Implementar triggers, views, stored procedures, functions e índices justificados;
- Definir controle de acesso com usuários distintos, sem uso do root;
- Criar IDs customizados para dados críticos, como consultas;
- Desenvolver API REST em Python/Flask que se comunica com o banco de dados;
- Construir interface web que exiba dados das views e permita agendamento via stored procedure.

3. METODOLOGIA

O desenvolvimento do sistema seguiu o ciclo de vida clássico de projetos de banco de dados: levantamento de requisitos, modelagem conceitual, modelagem lógica, modelagem física, implementação, integração e testes.

Na fase de levantamento, foram identificados os processos essenciais de uma clínica odontológica: cadastro de usuários (dentistas e pacientes), agendamento de consultas, controle de estoque e gestão financeira. A partir daí, definiu-se o Modelo Entidade-Relacionamento (MER), que evoluiu para o modelo lógico relacional e, finalmente, para o script DDL em MySQL.

Para o backend, utilizou-se Python com o microframework Flask, expondo endpoints REST que consomem o banco de dados via driver PyMySQL. O frontend foi desenvolvido com HTML5, CSS3 e JavaScript puro, garantindo leveza e compatibilidade. A comunicação entre as camadas segue o padrão cliente-servidor, com a interface consultando as views do banco de dados para exibição dos dados.

4. DESCRIÇÃO DO SISTEMA

4.1 Funcionalidades

- Dashboard com indicadores: consultas ativas, total de pacientes, receita paga e alertas de estoque;
- Agenda completa exibindo dados da view vw_agenda_completa;
- Agendamento de nova consulta via stored procedure sp_agendar_consulta;
- Listagem de pacientes com histórico hospitalar e endereço;
- Visualização do corpo clínico com CRO e especialização;
- Controle financeiro com status de pagamento (receitas e despesas);
- Controle de estoque com alertas de validade (vencido, atenção, OK);
- Relatório de faturamento por dentista via view vw_faturamento_dentistas.

4.2 Tecnologias Utilizadas

Camada	Tecnologia	Justificativa
Banco de Dados	MySQL 8.0	SGBD relacional robusto, amplamente adotado, com suporte completo a triggers, procedures, views e controle de acesso
Backend	Python 3 + Flask	Linguagem de alta legibilidade, Flask é leve e adequado para APIs REST em projetos acadêmicos e protótipos

Camada	Tecnologia	Justificativa
Conector BD	PyMySQL	Driver Python nativo para MySQL, sem dependências externas complexas
Frontend	HTML5 + CSS3 + JS	Tecnologias padrão da web; sem frameworks adicionais para reduzir complexidade de configuração
CORS	Flask-CORS	Necessário para comunicação entre frontend (porta 80) e backend (porta 5000) em ambiente local

5. MODELAGEM DO BANCO DE DADOS

5.1 Entidades e Relacionamentos

O banco de dados clinica_odontologica é composto por dez tabelas que modelam os processos clínicos, administrativos e financeiros da clínica. A tabela usuarios é a entidade-base do sistema, da qual herdam as especializações cliente e dentista, padrão conhecido como herança por delegação (tabela-pai + tabela-filha com FK compartilhando a PK).

Tabela	Descrição
grupos_usuarios	Agrupa usuários por perfil de acesso (Administrador, Dentista, Recepcionista)
usuarios	Dados básicos de todos os usuários do sistema (login, senha, nome, grupo)
cliente	Especialização de usuario com histórico hospitalar e endereço
dentista	Especialização de usuario com CRO e especialização clínica
consulta	Agendamentos com data, tipo, status, paciente e dentista responsável
procedimento	Catálogo de procedimentos odontológicos com valor
consulta_procedimento	Tabela associativa N:N entre consulta e procedimento
financeiro	Registros de receitas e despesas vinculados ou não a consultas
estoque	Controle de insumos com quantidade e validade
procedimento_estoque	Associação N:N entre procedimento e estoque (consumo)

5.2 Índices

Dois índices foram criados com justificativa técnica:

Índice	Tabela/Coluna	Justificativa
idx_consulta_data	consulta(data_hora)	Consultas de agenda filtram massivamente por data/hora. O índice reduz o custo de $O(N)$ para $O(\log N)$, essencial em operações frequentes de leitura da agenda
idx_financeiro_status	financeiro(status_pgto)	Relatórios financeiros filtram constantemente por status de pagamento (Pago/Pendente). O índice acelera JOINS e WHEREs sobre essa coluna

5.3 Triggers

Foram implementadas duas triggers com funções distintas:

Trigger	Evento	Função
tr_baixa_estoque_automatica	AFTER INSERT em consulta_procedimento	Quando um procedimento é vinculado a uma consulta, desconta automaticamente os insumos utilizados da tabela estoque, garantindo consistência sem intervenção manual
tr_impedir_alteracao_cancelada	BEFORE UPDATE em consulta	Impede qualquer alteração em consultas com status 'Cancelada', lançando erro via SIGNAL SQLSTATE, protegendo a integridade do histórico clínico

5.4 Views

Duas views foram criadas para simplificar consultas complexas e servir como interface de leitura para o frontend:

View	Descrição	Uso
vw_agenda_completa	JOINS entre consulta, usuarios (paciente e dentista) e financeiro, exibindo dados consolidados da agenda	Exibida diretamente na tela de Agenda do frontend; elimina a necessidade de múltiplas queries na camada de aplicação
vw_faturamento_dentistas	Agrega total de atendimentos e receita gerada por dentista, filtrando apenas pagamentos confirmados	Alimenta o gráfico de faturamento no Dashboard; concentra lógica de negócio no banco de dados

5.5 Stored Procedures e Functions

Objeto	Tipo	Descrição
fn gerar_id_consulta()	Function	Gera ID customizado no formato 'CON{ANO}-{SEQUENCIAL}' (ex.: CON2026-00001). Utiliza COUNT por ano para garantir unicidade e rastreabilidade temporal, sem AUTO_INCREMENT
sp_agendar_consulta(...)	Procedure	Encapsula a lógica de agendamento: chama fn gerar_id_consulta() e insere o registro em consulta. Garante que o ID seja sempre gerado de forma padronizada
sp_criar_usuario(...)	Procedure	Centraliza a criação de usuários com validação básica, facilitando futuras expansões como hash de senha

O uso de function para geração de ID foi escolhido para dados críticos (consultas) pois permite rastreabilidade temporal e padrão semântico. O AUTO_INCREMENT foi mantido para tabelas de suporte (grupos, procedimentos, estoque, financeiro), onde a rastreabilidade customizada não é necessária.

5.6 Usuários e Controle de Acesso

O acesso ao banco de dados é proibido via root. Foi criado o usuário `app_odontoped_user` com permissões granulares:

- SELECT, INSERT, UPDATE, DELETE no schema completo — utilizado pelo backend Flask para operações CRUD;
- SELECT exclusivo nas views `vw_agenda_completa` e `vw_faturamento_dentistas` — nível de acesso destinado a relatórios e consultas do frontend.

Os grupos de usuários da aplicação (tabela `grupos_usuarios`) definem três perfis: Administrador (acesso total ao sistema), Dentista (acesso a prontuários e agenda) e Recepcionista (acesso a cadastros e agendamentos). Essa separação permite implementar lógica de autorização na camada de aplicação com base no campo `id_grupo` da tabela `usuarios`.

6. CONCLUSÃO

O sistema de gestão para clínica odontológica foi desenvolvido com sucesso, contemplando todos os requisitos técnicos estabelecidos: estrutura relacional completa com dez tabelas, triggers de automatização e proteção de dados, views para abstração de consultas complexas, stored procedures e functions para encapsulamento de lógica de negócio, índices justificados para performance, e controle de acesso sem uso do root.

A integração entre o banco de dados MySQL, a API REST em Python/Flask e o frontend em HTML/CSS/JS demonstrou a viabilidade de construir soluções completas e funcionais com tecnologias abertas e acessíveis. O uso de mock data no frontend garante que a aplicação possa ser demonstrada mesmo sem conexão ativa ao banco, o que facilita apresentações.

Como trabalhos futuros, sugere-se a implementação de autenticação JWT na API, hash de senhas com bcrypt, paginação dos resultados, e possível migração do frontend para um framework como React ou Vue.js para maior escalabilidade.

7. REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2019.

DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Campus, 2004.

MySQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

FLASK. Flask Documentation. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: mai. 2026.

CONSELHO FEDERAL DE ODONTOLOGIA. Dados estatísticos. Disponível em: <https://cfo.org.br/>. Acesso em: mai. 2026.